



AURA

Interruptible Real-Time Agent

Intelligence that adapts to your latest intent.

Theme 05 – Interruptible Real-Time Agents

Team: MSRIT_An Apple a Day

College: MS Ramaiah Institute of Technology

Team Members:

- Udbhav Shukla
- Aditya Raj Chauhan
- Bhuma Lakshmi Hansika
- Eggönü Venkata Tejal

Theme 05 – Interruptible Real-Time Agents

Core Challenge:

"AI agents must remain responsive and correct even when users interrupt, revise, or correct their intent while work is already in progress."

Three Key Pillars:



FAST PATH

Immediate response, acknowledgement, and interruption detection. Keeps perceived latency low.



SLOW PATH

Reasoning, planning, and asynchronous tool execution. Handles complex tasks without blocking.



COORDINATION

State management, cancellation semantics, and re-planning. Ensures consistency across interruptions.



Why Do AI Assistants Fail When the User Changes Their Mind?

Traditional agents may continue executing an old request even after the user has issued a correction.



STALE RESULTS

Old asynchronous results arrive after the user has changed the request. The system accepts outdated information as current.



STATE CORRUPTION

The system may mix old and new intent, leading to inconsistent session state and confused behavior.



DUPLICATE SIDE EFFECTS

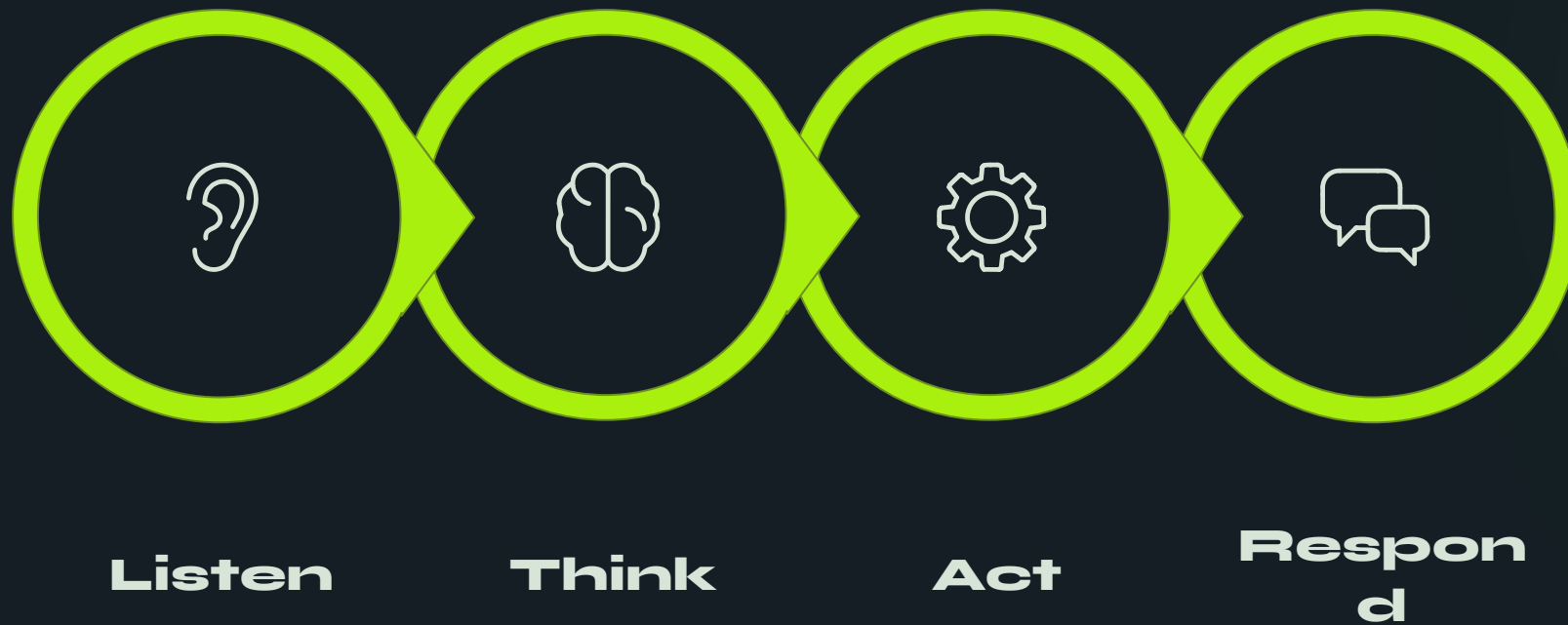
Repeated or retried state-changing tools can execute twice, causing unintended consequences (e.g., double bookings).



POOR INTERRUPTION HANDLING

The assistant treats interruption as another request instead of an execution event, leading to race conditions.

Existing Approaches & Their Gaps



Traditional sequential agent architectures follow a linear path, which can lead to significant challenges when dealing with real-time interruptions and dynamic user intent:

- Sequential execution blocks on long-running tasks
- In-flight work continues even after user correction
- Weak cancellation semantics (no explicit call tracking)
- No explicit state-version validation
- Late results can become misleading or incorrect
- Duplicate side effects are difficult to prevent
- No distinction between interruptible and commit-boundary operations

Typical sequential agent architectures can struggle with real-time interruptions and state consistency.

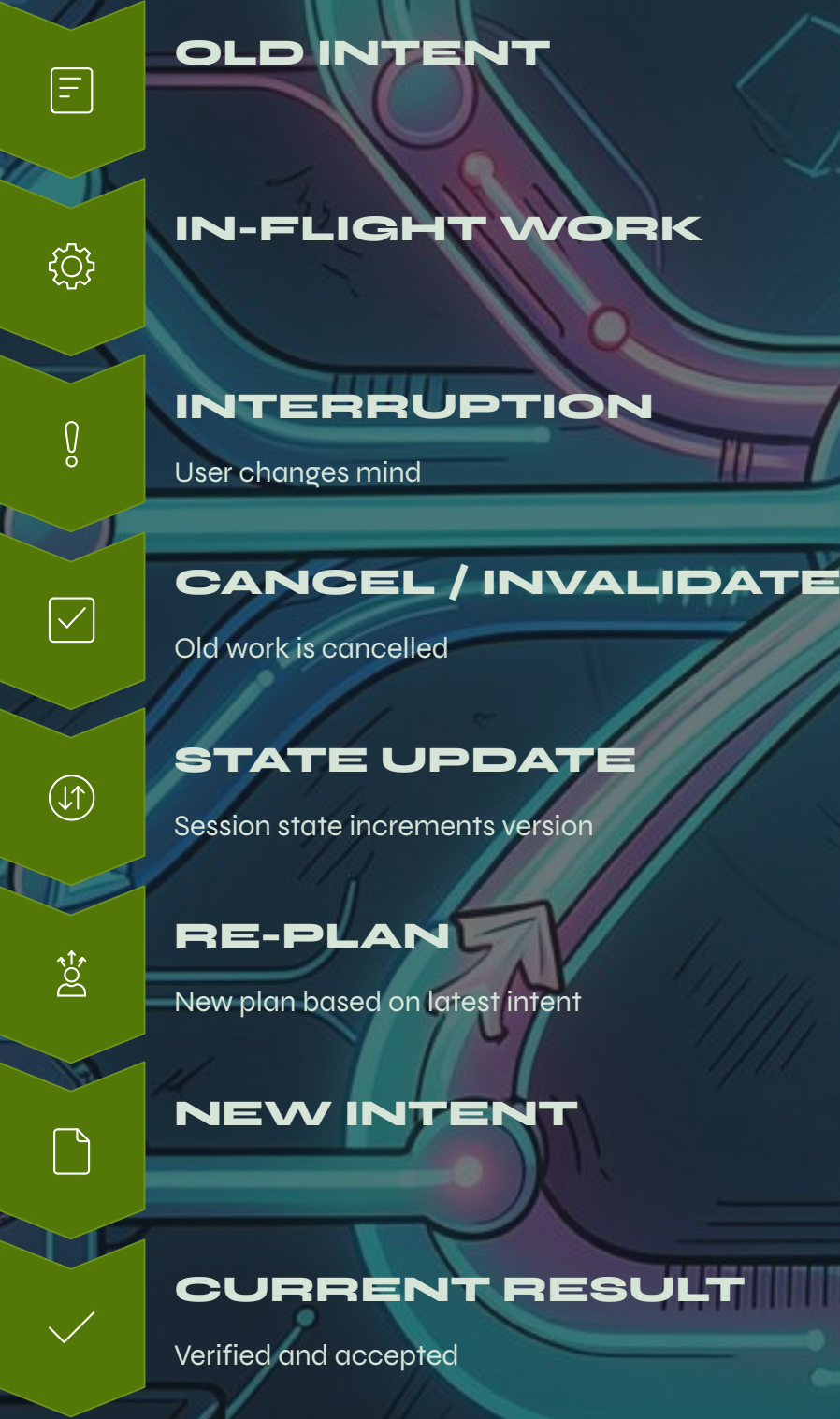
AURA – Latest Intent Wins

Core Principle:

"Latest Intent Wins."

The latest user correction becomes authoritative. Old work may still physically finish, but it cannot overwrite the current session state if it is stale.

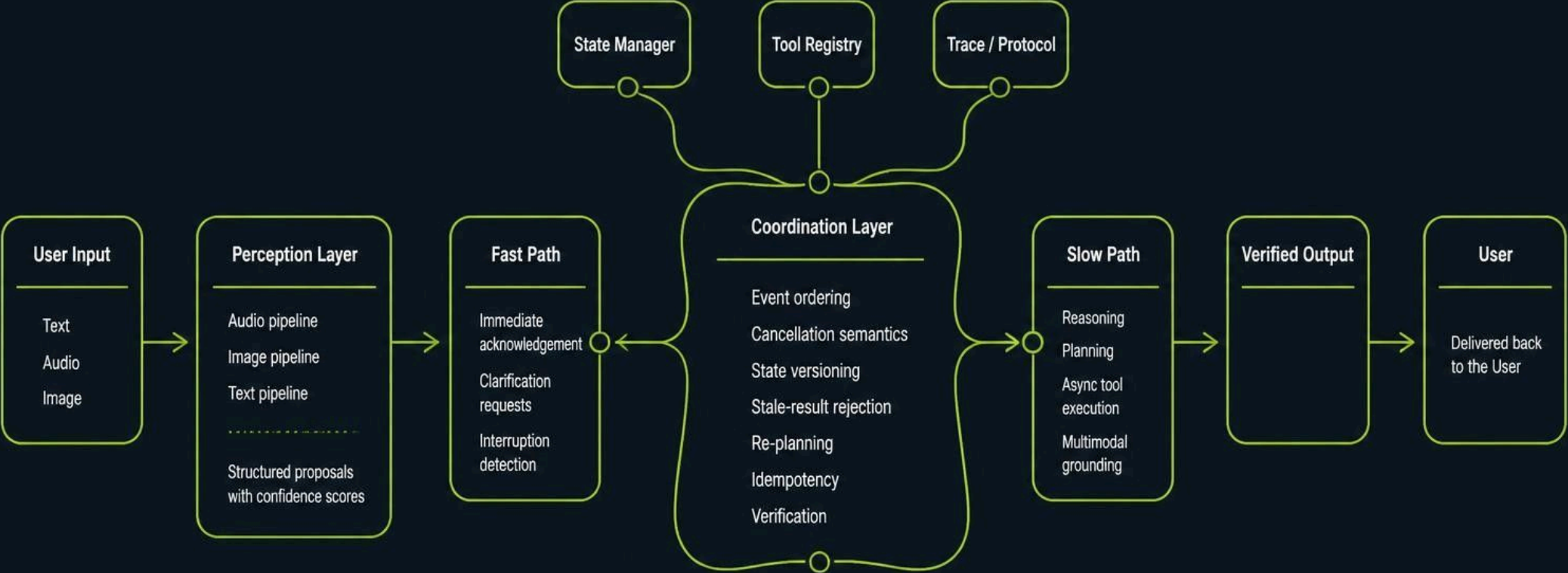
AURA Execution Flow:



Key Insight:

"AURA treats interruption as a first-class execution event, not an afterthought."

AURA Architecture



From User Input to Verified Output



USER INPUT

User provides request (text, audio, or image)



FAST ACK

Immediate acknowledgement and interruption detection



COORDINATION

Event ordering, state versioning, and cancellation



SLOW-PATH EXECUTION

Reasoning, planning, and async tool execution



VERIFICATION

Perception proposes → Verification commits



FINAL SNAPSHOT

Verified output delivered to user

"PERCEPTION PROPOSES → VERIFICATION COMMITS"

Live Prototype – Interrupt, Cancel, Re-plan

The AURA prototype demonstrates real-time interruption handling through a live web interface.

Key Features Visible in Demo:

LIVE AGENT PAGE

Real-time execution feed showing all tool calls, state changes, and user interactions.

EXECUTION TRACE

Detailed log of every event: call_id, state_version, timestamp, action type.

CURRENT STATE SNAPSHOT

Live display of the session state, showing all variables and their current values.

ACTIVITY FEED

Chronological list of all events: user input, fast ACK, coordination decisions, tool execution, result acceptance/rejection.

CALL TRACKING

Each tool call is tagged with unique call_id and state_version for traceability.

STALE vs CURRENT LABELS

Results are explicitly marked as STALE (rejected) or CURRENT (accepted) based on state versioning.

Note: Use placeholder frames if actual screenshots are not available.



Latest Intent Wins – The Bangalore → Mumbai Scenario

t₀ – USER REQUEST

User: "Find a flight from Bangalore to Delhi."

- Fast ACK received.
- call_oo1 starts
- state_version = 0
- destination = Delhi

t₃ – USER INTERRUPTS

User: "Actually Mumbai, not Delhi."

t₅ – COORDINATION RESPONSE

- call_oo1 cancelled
- State updated: v₀ → v₁
- destination: Delhi → Mumbai
- call_oo2 starts
- destination = Mumbai

t₇-t₈ – RESULT HANDLING

- call_oo1 result arrives
 - Status: STALE (version mismatch)
 - Result: REJECTED
- call_oo2 result arrives
 - Status: CURRENT (version match)
 - Result: ACCEPTED

FINAL STATE

- origin = Bangalore
- destination = Mumbai

State Consistency & Safe Tool Execution

State Consistency Rules

Every tool call is tagged with:

- session_id (unique session identifier)
- call_id (unique call identifier)
- state_version (current state version at call time)

Acceptance Rule:

- result.version == current.state_version → ACCEPT
- result.version < current.state_version → STALE → REJECT

State Updates:

- Delta-based (only the corrected slot changes)
- Before: origin = Bangalore, destination = Delhi
- After: origin = Bangalore, destination = Mumbai

Safe Tool Execution

Tool Semantics (Commit Classification):

- **READ / SEARCH**
 - → Fully interruptible
 - → No side effects
- **RESERVE / PREPARE**
 - → Cooperatively cancellable
 - → Can be rolled back
- **COMMIT / BOOK / PAY**
 - → Commit boundary
 - → Cannot be interrupted mid-transaction

Idempotency Keys:

Prevent duplicate side effects even if a tool is retried.

Cancellation:

Cooperative; commit boundaries are always respected.

Multimodal Grounding & Verification



Perception

Verification

Commit

"Perception does NOT directly mutate authoritative state."

Perception proposes.

Verification checks.

State commits.

Prototype supports deterministic audio harness and image/frame grounding.

Tools & Technology Stack



FRONTEND

- React
- TypeScript
- Vite
- React Router

Real-time control surface and execution feed.



CORE

- Python
- Asynchronous execution
- Fast Path / Slow Path separation
- State Manager
- Tool Registry
- Coordination Layer



PROTOCOL

- Structured JSON events
- Schema validation
- Unique call_id
- state_version tracking
- Trace logging



TESTING

- Unit/integration tests
- Deterministic mock providers
- Latency/failure injection
- Replayable traces

Where Interruptible Agents Matter



TRAVEL & BOOKING

Change destination, date, or preferences during an active flight search. User can interrupt and correct without losing context.



IN-CAR ASSISTANTS

Redirect navigation or modify a route while an action is running. Driver can change their mind safely without system confusion.



CUSTOMER SUPPORT

Correct or modify a workflow without restarting the conversation. Support agent can adjust the ticket or action mid-process.



TROUBLESHOOTING & DIAGNOSTICS

Combine live image/audio input with tool information. User can provide new information and the system re-plans immediately.



ACCESSIBILITY & HANDS-FREE INTERACTION

Support natural corrections and interruptions in voice-only or hands-free interaction. Users with accessibility needs can interrupt naturally.

Innovation Highlights, Results & Limitations

INNOVATION HIGHLIGHTS

Latest Intent Wins — New corrections become authoritative

Stale-Result Rejection — Old asynchronous results are explicitly rejected

Commit-Boundary Semantics —
READ/SEARCH/RESERVE/COMMIT classification

Perception → Verification → Commit — Multimodal grounding pipeline

Versioned Session State — Every state change is tracked

Cooperative Cancellation — Cancellation respects commit boundaries

Idempotent State-Changing Tools — Prevent duplicate side effects

Replayable Traces — Full observability and deterministic testing

RESULTS

Prototype capabilities verified through deterministic tests and scenario traces.

(Do NOT invent latency, accuracy, recovery, or performance numbers.)

LIMITATIONS

- Real provider APIs may require credentials/sandbox access
- Prototype uses deterministic mocks where real backends are unavailable
- Compensation/reconciliation is demonstrated as an interface where a real provider cannot safely be exercised
- The prototype does not claim to solve all security or distributed-consensus problems

What's Next

NOW

Interruptible execution prototype with deterministic testing and live UI.

NEXT (Medium-term)

Richer multimodal interaction (improved audio/image grounding, real-time feedback).

SCALE (Future)

Larger scenario/evaluation coverage and production deployment.

NEXT (Short-term)

Broader external tool integrations (real flight APIs, payment systems).

PRODUCTION (Long-term)

Distributed coordination and persistent session infrastructure.

Important Note:

Clearly distinguish future scope from implemented functionality.

Why AURA Is Technically Distinctive

1

LATEST INTENT WINS

New corrections become authoritative. Old work cannot overwrite current state if stale.

2

STALE RESULTS DON'T WIN

Old asynchronous results are explicitly rejected based on state versioning, not just ignored.

3

CANCELLATION HAS SEMANTICS

Cancellation respects commit boundaries. READ/SEARCH operations are fully interruptible; COMMIT operations are protected.

4

SIDE EFFECTS ARE PROTECTED

Idempotency keys prevent duplicate state changes even if a tool is retried or executed multiple times.

5

PERCEPTION DOESN'T OWN STATE

Perception proposes; verification checks; state commits. Clear separation of concerns prevents silent state corruption.

6

EVERYTHING IS TRACEABLE

Tool calls, state transitions, and events are fully observable through `call_id`, `state_version`, and trace logging.

Submission Readiness & Links

Final Checklist:

- ☐ Working prototype
 - ☐ Public GitHub repository
 - ☐ README / reproducible setup
 - ☐ Frontend prototype (React + TypeScript)
 - ☐ Backend / core architecture (Python)
 - ☐ Testing / evaluation infrastructure
 - ☐ Presentation (this deck)
 - ☐ Final release tag
-

THANK YOU

AURA

"Intelligence that adapts to your latest intent."

**LISTEN → ACK → ACT →
INTERRUPT →
CANCEL/RE-PLAN →
VERIFY → COMMIT**

Team: MSRIT_An Apple a Day

Theme: Theme 05 — Interruptible Real-Time Agents

College: MS Ramaiah Institute of Technology



[AURA – Interruptible Real-Time](#)



[GitHub - udbhavshukla/Interruptible-RealTime-Agent: An event-driven multimodal AI agent for real-time conversations, asynchronous tool execution, interruption recovery, and dynamic re-planning.](#)